L04

委托 Codex 执行一次最小变更

工作台 V0 实践

本节课如何实践 FDE

01 现场：确认库存核对需求
明确可用量、仓位与批次，沿用 L03 Spec

02 能力：按 AGENTS 与 Spec 补齐 V0
你监督 Codex，接通执行、范围检查与最小 Eval

03 能力验收：独立复验工作台（A）
验证越界能拦截、失败能留证，再接受具体版本

04 交付：由已接受的 V0 组织库存导出（B）
Codex 在授权范围实现；人核对 CSV 与库存状态

05 反馈：区分业务错误与工作台缺口
业务错误修 B；控制失效修 A，复验后再交付

FDE：Forward-Deployed Engineering
把现场问题变成可验收交付，并积累可复用能力。

本讲目录

01 **看见工作台与交付目标**

02 用具体输入接通工作台 V0

03 在页面中复验并接受 Ticket A

04 让 V0 组织库存导出

05 用数据复验、返工与迁移

建设产品

个人研发工作台 V0



先补齐任务接收、执行与检查，
再用它交付 FlowERP 库存导出。

参考界面含后续能力，非本讲全部建设范围。

这就是组织研发的工作台



页面使用流程

登记问题

↓

确认范围

↓

查看交付记录

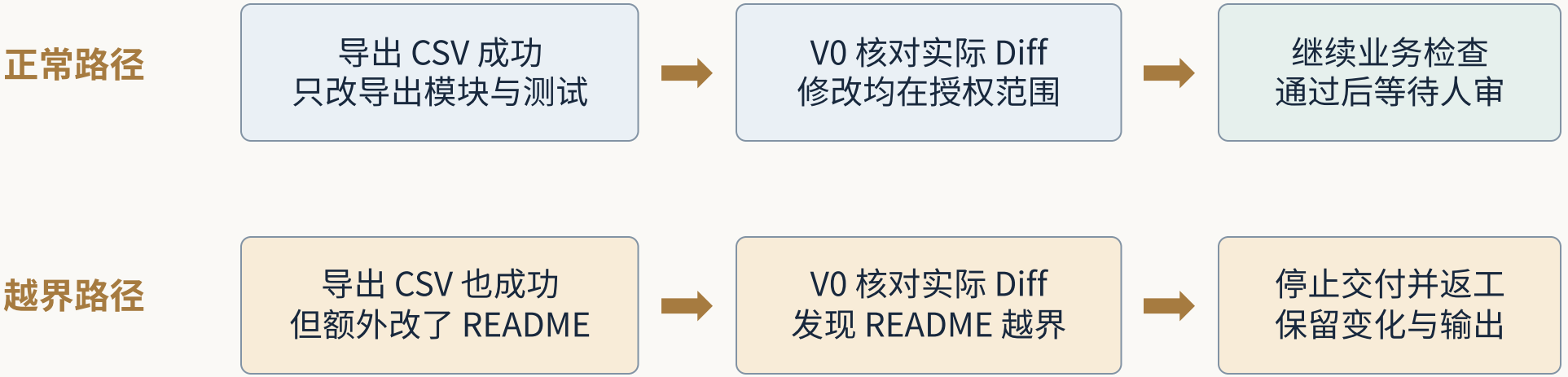
↓

核对结果

2026-09-07 原生界面；参考首页当时没有事项，不代表已贯通 A。

CSV 导出成功，但修改了未授权文件，仍须返工

教学对照：本次只允许修改导出模块和对应测试



先用两条路径验收 V0：越界要拦住，正常不能误拦。独立接受后，再用它交付库存导出。

本讲目录

01 看见工作台与交付目标

02 用具体输入接通工作台 V0

03 在页面中复验并接受 Ticket A

04 让 V0 组织库存导出

05 用数据复验、返工与迁移

工作台收到 Spec，
如何校验输入、
限制修改范围，
并在失败时停止？

给 Codex 的任务说明：补齐工作台 V0

Spec 就是任务说明：写清目标、边界和检查方法。下面是一份教学示例。

六段内容	本次 V0 建设任务怎么写
来源：为什么做	已有 Spec 解析器，还需要接通执行与检查。
目标：做成什么	让工作台接收 Spec、调用 Codex、检查并保存结果。
非目标：这次不做	暂不开发库存导出，先把工作台建好。
约束：守哪些边界	只改确认的模块与测试，失败时保留输出和变化。
验收用例：怎样检查	缺字段不执行；越界判失败；正常通过后等待人审。
完成定义：何时算完成	同伴复验并接受当前 V0 版本，再开始库存导出。

你先确认任务，再委托 Codex。下一页看：缺少“完成定义”时，解析器如何报错。

缺少完成标准，工作台应先退回任务说明

1 输入缺了什么

已有：来源、目标、
非目标、约束、验收用例。

缺少：完成定义。

没有写清结束条件，
无法判断任务何时完成。

2 解析器实际报错

```
ValueError: Spec 缺少必要章节：完成定义
```

缺少章节：完成定义
退出码：1（输入不完整）

实际观察：解析器拒绝输入。
执行器是否未启动，
还需要控制检查。

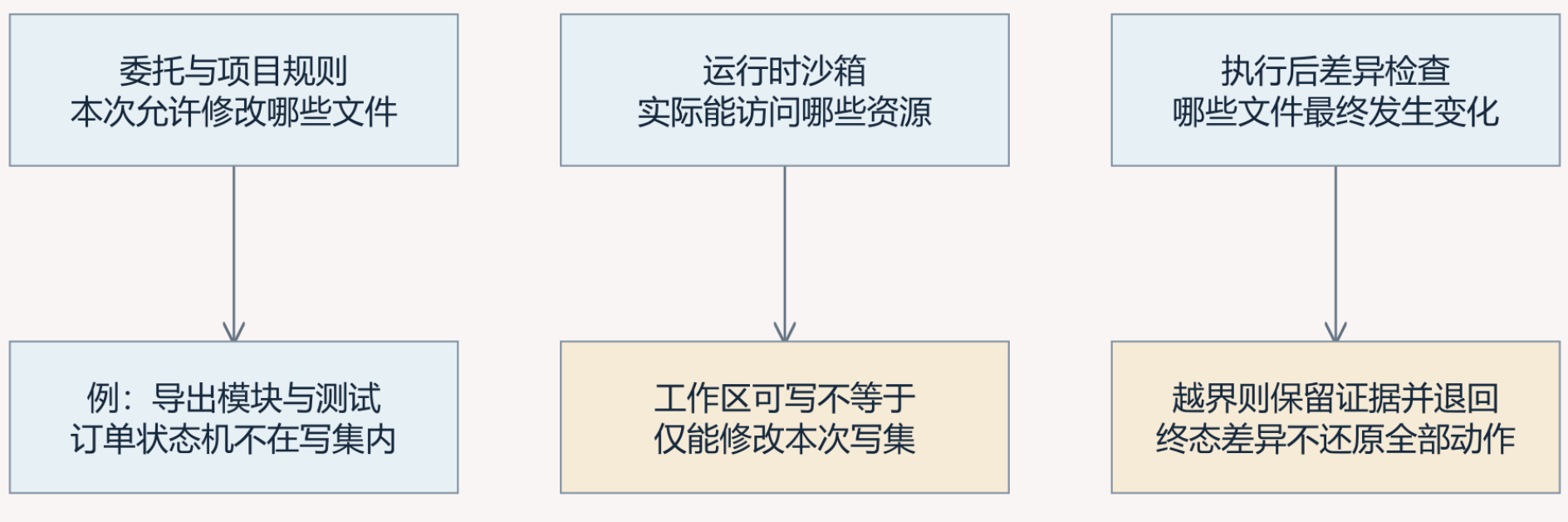
3 补全后再检查

补写完成条件，例如：
同伴复验并接受 V0，
才算完成本次建设。

重新解析应成功。
再验证：缺字段时，
执行器没有被调用。

操作顺序：修正任务说明，再重新提交。不能删掉校验规则，让不完整的输入继续执行。

执行边界：约定、权限、实际变化



例如：授权导出模块和测试，不代表可以顺手修改订单状态机。

Codex 退出 0，修改就合格吗？

本次实际变化（教学示例）	事先允许？	判断
修改 flowerp/export.py	是	继续检查业务
新增 tests/test_export.py	是	核对断言
修改 README.md	否	列出越界，退回
新增 .env	否	阻止提交，保护凭据

教学文件名。真实执行缺写集应先拒绝；越界失败后，也要补跑范围内正常路径。

一次变更怎样从执行结束走到人工接受

三步始终核对同一份实际改动。前一步通过，才具备进入下一步的条件。

① 执行是否正常结束？

工作台读取 Codex 执行记录

看退出码、超时和异常。
退出码 0 只说明进程正常结束，
还不能证明改动符合要求。

正常

② 实际改动符合要求吗？

独立复验者借助工作台检查

对照 Spec 看改动范围，
运行最小 Eval，核对实际输出。
通过只覆盖本次检查的内容。

通过

③ 审核者接受这个版本吗？

具名审核者作出决定

核对同一版本的改动与报告。
判断需求与剩余风险。
明确接受后才进入后续交付。

异常或超时：停止本次交付

保留日志和部分结果。
排错后重新执行，再做检查。

检查失败：退回修订

说明哪项要求未满足。
修订后重新检查新版本。

未接受：区分等待与返工

无人决定：保持待审核。
明确拒绝：按理由返工并复验。

例：退出码 0，但多改了未授权的 README

第①步正常，第②步因越界失败，退回修订。不能把“执行成功”直接当作“已接受”。

本讲目录

01 看见工作台与交付目标

02 用具体输入接通工作台 V0

03 在页面中复验并接受 Ticket A

04 让 V0 组织库存导出

05 用数据复验、返工与迁移

本节操作：验收工作台 V0

① 首页：事项与决策
找到“建设工作台 V0”的事项

↓

② 交付记录：打开任务 A
核对任务编号、Spec 与实际改动

↓

③ 同伴独立复验同一版本
查看检查报告，说明剩余风险

↓

④ 具名接受有效的 A
再提交 B：交付库存导出

检查失败或未获接受：留在 A 处理。
修订后重新复验，再作接受决定。

事项记录保留问题与决定，任务记录保留执行证据



对着表单填写：补齐工作台 V0

所属项目：自己的工作台。
这次想解决什么：让越界修改被识别并停止。
描述问题：执行器报告成功，却额外改了 README。
已有判断：只补范围检查及测试，暂不扩展业务。

为什么不能只写“帮我完善工作台”？

没有具体失败，无法判断修复是否有效；
没有范围决定，Codex 容易把“完善”理解成扩功能。

保存后怎样继续

用事项说明形成能力 Spec，再记录建设与复验任务。若页面未自动关联，写入真实任务编号与证据位置。

检查通过仍待人审，因为检查不能替人决定接受范围



图中三条信息分别说明什么

“仅复验”：这次运行检查，没有调用 Codex 编码。
“检查通过”：已有用例的断言成立。
“等待具名人审”：尚未有人接受这个版本。

反例：终态参考仓库本来就是绿的

看到参考任务的绿灯，就声称“我已建成 V0”。
页面既未证明本人补齐了什么，也未证明同伴复验了同一版本。

接受前必须回查	为什么要查
建设 Diff 与 Spec	确认实际增量及约定范围
同一候选的独立复验、剩余风险	防止自签、检查错目录或把漏验当通过

绿色用例覆盖了什么？要看输入和断言



以 inventory_export_is_stable 为例

源码实际检查	可以支持的结论
两种商品，数量 3 与 2，预占均为 0	该组输入的可用量为 3 与 2
八列表头、A 在 B 前、行末数量	该组数据的字段和商品排序正确
再次读取两种商品的 available	这两个可用量没有被导出改动

为什么这盏绿灯还不够？

若错误实现直接把在库量当可用量，预占为 0 时仍能通过。把输入改为“在库 8、预占 3”，预期应为 5，才可能揭示这个错误。

下一步：补充非零预占、仓位／批次、特殊字符与失败后状态的检查；逐项记录真实结果。

B 被拒绝：还没有证据证明这套 V0 可以交付

```
cli.py: error: 工作台 Ticket A 尚未被具名接受
```

这次实测发生了什么

引用 A 请求真实执行，入口返回退出码 2：
“工作台 Ticket A 尚未被具名接受”。
此请求在资格检查处停止，B 编码没有启动。

为什么必须在编码前检查？

B 依赖 V0 限制改动并保存结果。
若这些能力尚未被独立验证，
即使 CSV 生成了，也无法据此
信任它的范围检查与交付记录。

入口核对 A 的条件	要防止什么
能力任务身份、具名接受	借用无关任务或未完成验收
非构建者复验、阻断检查通过	执行者自签或带失败继续
同一工作区、控制源码一致	借用另一目录或旧版本的接受

接受的是受检版本，控制源码改变就要重新复验

教学推演：上午通过，下午改代码

上午：版本 H1

同伴检查越界与超时分支，
接受 H1。

→

下午：版本 H2

修改范围检查函数，
控制源码指纹改变。

→

再次申请执行 B

H2 与受检 H1 不一致，
旧接受不能继续沿用。

为什么不能继续用上午的绿灯？

检查结论只对应当时的代码。
即使只改一行，也可能让越界检测失效。
源码指纹用于发现变化，本身不证明正确。

正确的恢复步骤

查看变化，复验当前控制源码；
按需重启，让服务运行新代码；
由独立复验者重新接受，再提交 B。

边界：运行记录增加不等于控制源码改变；换到另一工作区，也不能直接借用原接受。

本讲目录

01 看见工作台与交付目标

02 用具体输入接通工作台 V0

03 在页面中复验并接受 Ticket A

04 让 V0 组织库存导出

05 用数据复验、返工与迁移

A 有效后，
怎样把一次业务
委托交给 V0？

任务 B：让工作台交付库存明细 CSV

前提：工作台 V0 已通过独立验收且仍有效。本页示范怎样填写业务任务 B。

要交代什么	库存导出任务的写法（教学示例）	为什么需要这一项
按什么做	采用 L03 确认版 Spec，导出八列库存明细。	让执行者按已确认的需求实现。
在哪里改	在本次独立代码副本中修改并运行检查。	避免改了新代码，却验证旧仓库。
允许改什么	先查清调用路径，再列出导出实现与测试文件。	订单、采购和页面样式不在本次范围。
怎样验收	检查字段、排序、数量及导出前后的库存状态。	文件生成了，还要证明内容正确且只读。
何时停止	输入无效、修改越界或关键检查失败时停止交付。	保留失败现场，带着具体原因返工。

把“导出正确”变成可以核对的结果

例：在库 8、预占 3，CSV 的可用量应为 5。导出后，库存仍是 8 和 3。

执行前补齐真实 Spec 版本、绝对目录和逐文件清单，再交给工作台调用 Codex。

真实案例：工作台调用 Codex 修正可用库存

2026-09-08 隔离实测：把“可用量 = 在库量”修正为“在库量 − 预占量”。

执行器隔离实测 • 2026-09-08

修改前：独立检查

```
AssertionError: expected 5, got 8
```

真实 Codex 进程

```
returncode: 0    changed_files: ["inventory.py"]
```

实际文件差异

```
--- a/inventory.py
+++ b/inventory.py
@@ -1,2 +1,2 @@
 def available(on_hand, reserved):
-    return on_hand
+    return on_hand - reserved
```

修改后：独立检查

```
PASS: available(8,3)=5; available(8,0)=8
```

真实记录摘录的浏览器截图，非工作台原生页面。完整输入、事件与输出另存。

执行器机制实测，未交付完整 CSV，也未作人工接受。

这次怎样启动

工作台执行器调用
codex exec，
把任务从标准输入送入。

怎样限制与留证

--cd 指向隔离副本。
只准改 inventory.py。
--json 留下执行事件。

从截图读出的结果

仅改 1 个文件，退出 0。
随后独立检查两项通过。

本讲目录

01 看见工作台与交付目标

02 用具体输入接通工作台 V0

03 在页面中复验并接受 Ticket A

04 让 V0 组织库存导出

05 用数据复验、返工与迁移

拿具体数据检查导出结果，再判断业务与工具返工

验收练习：两仓同一商品，导出后应该怎样排列？

场景：同一种螺丝分别存放在东仓和西仓。现在要导出库存明细，供业务人员逐仓核对。

操作：在独立练习数据中，故意先放西仓、再放东仓，然后调用本次候选中的导出功能。

输入次序	商品	仓库 / 仓位 / 批次	在库	预占
第 1 条	P001 螺丝	WEST / B01 / LOT02	12	4
第 2 条	P001 螺丝	EAST / A01 / LOT01	8	3

为什么故意倒序？

若按输入顺序原样输出，西仓仍在前，就能发现程序没有按合同排序。

正确结果：东仓 EAST 在前、西仓 WEST 在后。两条都保留，可用量分别是 8－3=5、12－4=8。

同一商品也不能合并成一行“可用量 13”，否则业务人员看不到货在什么仓位和批次。

教学输入，非生产数据。合同按商品、仓库、仓位、批次升序排列；下一页把预期结果展开成 CSV。

库存明细怎样导出，又怎样逐行验收？

① 输入：沿用上一页

先西仓：在库 12、预占 4。
后东仓：在库 8、预占 3。

② 程序：生成明细 CSV

按合同排序，逐行减去预占。
保留两条明细，不合并商品。

③ 同伴：读回实际文件

用 CSV 读取器解析字段。
与下方预期逐列、逐行比较。

预期答案：第一行是表头，后面两行分别对应东仓和西仓

sku	name	site	location	lot_id	on_hand	reserved	available
P001	螺丝	EAST	A01	LOT01	8	3	5
P001	螺丝	WEST	B01	LOT02	12	4	8

怎样判断这份文件正确？

表头恰好八列且次序一致。东仓在前、西仓在后，仓位和批次仍与各自的数量对应。

两行可用量分别为 5 和 8。只找到字符“5”，或只核对合计 13，都不能证明明细正确。

教学预期输出，非实际运行结果。实践需保存真实 CSV；后续还要检查导出未改变库存状态。

CSV 已生成，发现错误后怎样返工？

沿用前两页数据。下面是三种独立的教学反例，用来练习发现“文件生成成功”背后的错误。

① 同伴核对实际 CSV
按合同检查表头、行数与逐行数值

② 不符合要求就退回 B
保存输入、错误文件和预期差异

③ Codex 修订后再复验
在新版本运行原反例及另一组数据

假设读到了什么	合同要求是什么	据此写出的返工理由
东仓可用量为 8	东仓应为 8 − 3 = 5	漏减预占，修正可用量计算
只有一行，可用量为 13	东仓 5、西仓 8，保留两条明细	合并丢失仓位和批次，恢复明细
表头以 name,sku 开头	前两列应为 sku,name	列次序错误，按合同恢复八列顺序

返工说明示例：让执行者拿到可复现的问题

“东仓在库 8、预占 3，CSV 实际输出 8，预期为 5。附输入与错误文件，请在 B 的范围内修复并复验。”

空库存、名称带逗号时，怎样检查 CSV?

普通商品导出正确，还不能证明空库存和特殊名称也能正确处理。下面三种情境应分别测试。

① 在练习数据中准备输入
分别测试空库存与特殊名称

→

② 调用候选中的导出功能
每种情境各自保存实际 CSV

→

③ 用 CSV 读取器读回
比较记录数、字段数和原始名称

准备的输入	读回后应当看到什么	为什么这样检查
库存明细为 0 条	保留八列表头，数据记录为 0 条	无数据仍应有结构，不能补一条假库存。
商品名称为“螺丝,镀锌”	名称仍是一个字段，每条记录仍有八列	名称中的逗号不应把后续字段挤错列。
名称含双引号或换行	名称与原文一致，记录数与输入一致	一个字段可含换行，不能按文本行数计数。

带逗号名称的正确编码示例

P001,"螺丝,镀锌",EAST,A01,LOT01,8,3,5

教学预期，非实际运行结果。用 CSV 解析器处理引号与分隔符，再逐字段比较，不要直接按逗号切开。

只读验收：导出 CSV 后，库存账也要检查

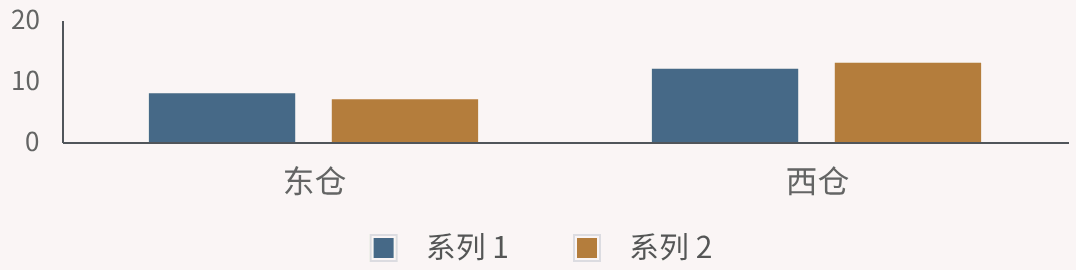
业务要求：导出只生成文件，不应改动库存账。练习时排除其他写入，才能判断变化是否由导出引起。

① 导出前：保存状态
记录每条库存及已有流水内容

② 执行：调用导出
分别测试成功与写出失败

③ 导出后：再次读取
逐条比较数量及流水是否改变

教学反例：比较库存账中的在库量（件）



总量相等，为什么仍要返工？

合计：8 + 12 = 7 + 13 = 20。
逐仓：东仓少 1，西仓多 1。
导出改变了库存，违反只读要求。

正确结果应是什么？

东仓仍为 8，西仓仍为 12。
原有库存流水内容也保持不变。

发现变化：保留前后状态和执行记录，退回 B 排查写入，再复验。CSV 内容正确也不能抵消库存账被改动。

导出中途失败，怎样证明旧文件没有被损坏？

场景：inventory.csv 已是可用的旧报表。更新导出写到一半出错，应保住旧报表并明确报告失败。

① 失败前：保存基准
保存旧文件内容和库存状态

→

② 测试中：制造中途失败
让写入器写出部分内容后抛出异常

→

③ 失败后：检查现场
核对失败记录、目标文件和库存

要检查的对象	失败后的正确状态	为什么不能只看“报错了”
原本有旧文件	目标文件与保存的旧内容逐字节相同	程序可能先清空旧文件，再写入失败。
原本没有目标文件	不留下可被当成完整报表的目标半成品	半份 CSV 可能被后续人员误用。
库存记录及原有流水	与失败前逐条比较，内容保持不变	文件报错，也可能已经改动库存账。

为什么故意在写入中途失败？

“目录不存在”只验证开始前的路径错误，无法发现旧文件已被截断的风险。中途失败才会检验保护是否有效。

教学故障实验，仅在临时练习目录进行。保留真实失败证据，修复后重跑相同失败路径；不得把失败记成成功。

返工先定位：业务做错了，还是工作台没有管住？

A：建设并验收工作台 V0。B：用这套工作台交付库存导出。两者的修复对象和继续条件不同。

发现问题后，先核对工作台的范围检查、失败记录和版本是否有效

控制有效，只有业务结果不符合要求

回到 B：修库存导出

例：东仓可用量输出 8，合同要求 5。
在 B 的批准范围内修正计算或明细处理。
复验新候选，交由审核者决定是否接受。

控制失效，或控制源码版本已改变

先回 A：恢复可信的控制与验收

例：改动越界，工作台却继续放行。
暂停 B，修复控制或复验已改变的版本。
A 经独立复验并重新接受后，再继续 B。

两类问题同时存在时，先处理 A

控制不可信时，继续修业务仍可能越界或丢失失败证据。控制源码改变也要重新复验，旧接受不能直接沿用。

先保存原始 Diff、错误 CSV 和检查报告。尚未定位原因时先调查，不能仅凭“测试红了”就判定 A 或 B。

skill 怎样在下一项任务中真正复用

当前阶段	给 skill 的具体材料	学生自己判断
建设 V0	AGENTS、能力 Spec、真实缺口	本次补齐什么
复验 A	控制版本、正反检查、实际输出	是否具备独立接受依据
交付 B	有效 A、业务 Spec、候选与写集	是否满足用途和边界
迁移采购导出	新状态口径、权限与字段	哪些检查要重新设计

采用 controlled-delivery-handoff 的具体版本，记录后续实际结果；文件存在不等于已复用。

本讲总结：把一个小需求交给工作台完成

学会的方法：用 Spec 明确任务，让工作台组织执行，再依据实际结果验收。

掌握的知识	什么时候用	具体怎样用
Spec 与任务绑定	编码前，避免做错需求	写清目标、边界和验收；保存采用的版本。
受控执行	委托 Codex 修改代码时	指定候选目录与允许文件；记录输出，失败就停。
Diff 与最小 Eval	编码后，判断改动是否合格	在同一候选核对实际改动，再检查业务结果。
独立验收	决定能否接受本次交付时	回查原始证据，说明接受理由或返工要求。

换个需求：导出未完成的采购申请

重新确认状态与字段，沿用已接受的 V0，限定改动，再检查文件和只读状态。

下一讲：裁判自己也可能判断错误

教学示例

导出程序把“可用量”写成“在库量”。
测试也把预期值写成“在库量”。

运行结果：测试全绿。

你会怎样判断？

怎样证明检查
真的能发现目标错误？

L05：先设计失败，再编写 Eval。用重复入库反例验证检查的辨别能力。